

DESIGN AND ANALYSIS OF ALGORITHMS

Instructor: Dr. Mert Büyükdede



Bubble Sort

It is one of the sorting algorithms developed to keep data in order in memory.

It sorts by simply arranging two consecutive memory blocks relative to each other and repeating this process continuously.

It is called bubble because it looks at two consecutive memory blocks.

Because this looking process moves on numbers like a bubble moves.



How does **BUBBLE SORT** work ?

1. Start from the beginning of the list.
2. Compare the first two elements.
3. If the first is greater than the second, **swap** them.
4. Move to the next pair and repeat the comparison and swap if necessary.
5. Continue to the end of the list. This completes **one pass**.
6. Repeat the process for the remaining elements (excluding the last sorted elements).
7. Continue until no more swaps are needed.

Bubble Sort

Our data that we want to rank:

5,7,2,9,6,1,3

Sorts the array after n repeats.

Step 1: 5,2,7,6,1,3,9

Step 2: 2,5,6,1,3,7,9

Step 3: 2,5,1,3,6,7,9

Step 4: 2,1,3,5,6,7,9

Step 5: 1,2,3,5,6,7,9

Step 6: 1,2,3,5,6,7,9



Bubble Sort

The working method of the algorithm can be explained as follows.

1. Take the first two numbers
2. Compare the two numbers you took
3. Write the smaller one and keep the other one in mind
4. If you have reached the end of the series, finish by writing the number in your mind in the series
5. If it is not the end of the series, take a new number.
6. Go back to step 2.



Let's try to create Bubble Sort algorithm

```
procedure bubbleSort(A : list of sortable items)
  n := length(A)
  repeat
    swapped := false
    for i := 1 to n-1 inclusive do
      { if this pair is out of order }
      if A[i-1] > A[i] then
        { swap them and remember something changed }
        swap(A[i-1], A[i])
        swapped := true
      end if
    end for
  until not swapped
end procedure
```

Let's try to java Bubble Sort algorithms

```
public void bubblesort(int[] A) // bir diziyi para
{
    int tmp;

    for (int i = 0; i < A.length; i++)
    {
        // for (int j = 1; j < A.length - i + 1; j++)
        for (int j = A.length - 1; j > i; j--) //
        {
            if (A[j - 1] > A[j])
            {
                tmp = A[j - 1];
                A[j - 1] = A[j];
                A[j] = tmp;
            }
        }
    }
}
```

When we look at the complexity of the algorithm, we see that each transition must pass through all elements at least once. In this case, each transition requires n operations, and since a total of $n-1$ transitions are required, it requires $n \times (n-1)$ transitions.

As a result, it has $O(n^2)$ time complexity. When we look at its memory requirement, it is enough to take up as much space as the current data. In this case, the memory complexity can be calculated as $O(n)$.

Bubble Sort Improvements

Bubble sort can be written as above, but two important optimizations can be made. For example, if we consider the code above and give an already sorted array to the loop, the loop will loop for the already sorted array, as long as the size of the array. Instead, we can prevent the array from looping after it is sorted.

So how do we know that the array is sorted?

If the array is sorted, the if condition that requires the elements to be swapped is never entered. In this case, if we do not enter the if in our code, the array is sorted and there is no need for further looping.



```
public void bubblesort(int[] A) // bir diziyi parametre alan fonksiyon
{
    int tmp;

    for (int i = 0; i < A.length; i++)
    {
        int sirali = 1; // Dizinin sıralı olduğunu varsayıyoruz

        for (int j = A.length - 1; j > 0; j--)
        {
            if (A[j - 1] > A[j]) // Şayet buraya girmiyorsak, dizi sıralı demektir
            {
                sirali = 0; // Şayet giriyorsak, sıralı değil demektir
                tmp = A[j - 1];
                A[j - 1] = A[j];
                A[j] = tmp;
            }
        }

        if (sirali) // Şayet dizinin üstünden geçtiğimiz halde
                    // hiçbir değer yer değiştirmiyorsa,
                    // dizi sıralıdır, döngüden çıkılabilir
        {
            break;
        }
    }
}
```

```

public class Main {
    public static void main(String[] args) {
        int[] A = {5, 3, 8, 4, 2}; // Örnek dizi
        bubblesort(A); // Metodu çağır

        System.out.println("Sıralanmış dizi:");
        for (int num : A) {
            System.out.print(num + " ");
        }

        public static void bubblesort(int[] A) {
            int n = A.length;
            for (int i = 0; i < n - 1; i++) {
                for (int j = 0; j < n - i - 1; j++) {
                    if (A[j] > A[j + 1]) {
                        int temp = A[j];
                        A[j] = A[j + 1];
                        A[j + 1] = temp;
                    }
                }
            }
        }
    }
}

```

In the new code left side, if you pay attention, the inner loop is returned as incomplete as the outer loop variable. Therefore, 0 will be returned in the 0th pass, 1 in the 1st pass and n elements will be returned incomplete in the nth pass. Because in the nth pass, the last n elements will already be sorted and will not need to be checked.

In this case, if we analyze the algorithm performance of our bubble sort again,

Best case analysis: n because if the entire array is already sorted, the loop is broken by looking at each element in the array once and the array is not passed again.

Worst case analysis: n^2 . Let's explain the reason as follows. In each pass, the number of elements should be looked at as many times as the number of passes. For example, for an array with n elements, n-k elements are looked at in the k th pass. Therefore, n elements are looked at in the 0th pass, n-1 elements are looked at in the 1st pass and n-n, that is, 0 elements are looked at in the last pass. In this case, the total number is the sum of the numbers from 1 to n and

$$n \times (n+1) / 2$$

the element needs to be looked at. In this case, the value $O(n^2)$ is found (since it is upper bound).

Bubble Sort

🧠 Key Points:

- Easy to understand and implement.
- Not efficient for large datasets.
- Time complexity:
 - Worst case: $O(n^2)$
 - Best case (already sorted): $O(n)$ (with optimization)
- Space complexity: $O(1)$ (in-place sorting)

Activity: Baggage Line Crisis Sorting Luggage with Bubble Sort

🧠 Scenario (Real-Life Context)

At a busy international airport, luggage on the conveyor belt has been randomly mixed. A robotic system must organize the suitcases in **ascending order of their weights** to safely load them into the cargo bay.



The robot is restricted to **comparing and swapping only two adjacent bags at a time**.

Your mission

Write a program to sort the baggage according to these rules.

Activity: Baggage Line Crisis Sorting Luggage with Bubble Sort

STEP I

Students will:

- Define an integer array representing luggage weights
 - `int[] bags = {23, 12, 45, 9, 18, 30, 25, 15};`
- Print each comparison and swap step
- Count and display total comparisons and swaps
- Include an **early-exit optimization** to improve efficiency



Required Components:

- `bubbleSort()` function
- `printArray()` helper method
- `main()` method to run the scenario

Activity: Baggage Line Crisis Sorting Luggage with Bubble Sort

STEP 2

Students will test their code with:

- A sorted array (best-case scenario)
- A reverse-sorted array (worst-case scenario)
- A random array (average-case scenario)
- Observe and record the number of passes, comparisons, and swaps
- Test if the early-exit optimization works correctly



```
public class BaggageSorter {

    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        int totalComparisons = 0;
        int totalSwaps = 0;
        boolean swapped;

        System.out.println("Initial luggage weights:");
        printArray(arr);
        System.out.println("Sorting process:");

        for (int i = 0; i < n - 1; i++) {
            swapped = false;
            for (int j = 0; j < n - i - 1; j++) {
                totalComparisons++;
                System.out.printf("Comparing %d and %d... ", arr[j], arr[j + 1]);
                if (arr[j] > arr[j + 1]) {
                    // swap arr[j] and arr[j+1]
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    totalSwaps++;
                    swapped = true;
                    System.out.println("Swapped.");
                } else {
                    System.out.println("No swap.");
                }
            }
            printArray(arr);

            if (!swapped) {
                System.out.println("Array already sorted. Early exit.\n");
                break;
            }
        }
    }
}
```

```
        System.out.println("Sorted luggage weights:");  
        printArray(arr);  
        System.out.println("Total comparisons: " + totalComparisons);  
        System.out.println("Total swaps: " + totalSwaps);  
    }  
  
    public static void printArray(int[] arr) {  
        for (int weight : arr) {  
            System.out.print(weight + " ");  
        }  
        System.out.println();  
    }  
  
    public static void main(String[] args) {  
        int[] bags = {23, 12, 45, 9, 18, 30, 25, 15};  
        bubbleSort(bags);  
    }  
}
```

Quick Sort

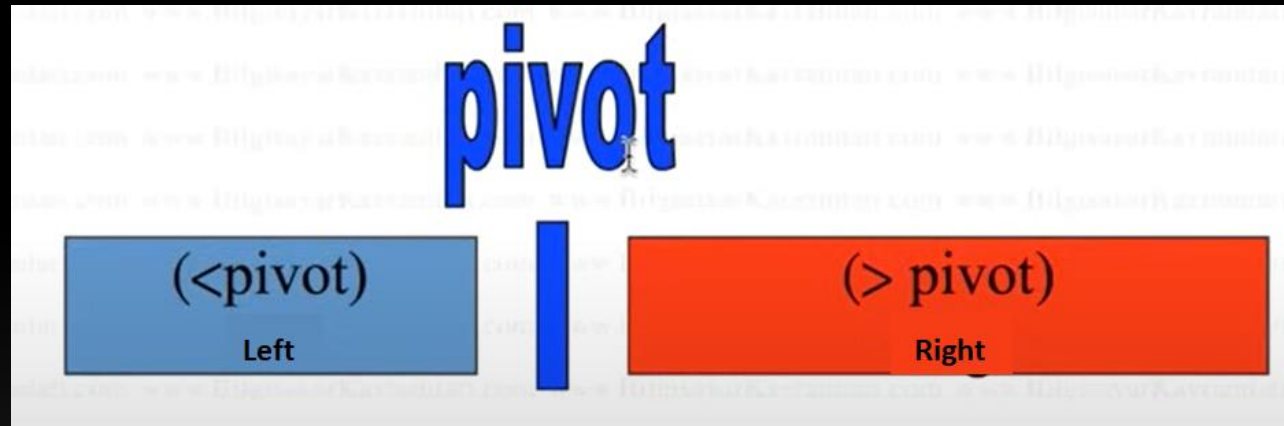
Quicksort is the other important sorting algorithm that is based on the divide-and-conquer approach. Unlike mergesort, which divides its input elements according to their position in the array, quicksort divides (or partitions) them according to their value.

A partition is an arrangement of the array's elements so that all the elements to the left of some element $A[s]$ are less than or equal to $A[s]$, and all the elements to the right of $A[s]$ are greater than or equal to it:

$$\underbrace{A[0] \dots A[s-1]}_{\text{all are } \leq A[s]} \quad A[s] \quad \underbrace{A[s+1] \dots A[n-1]}_{\text{all are } \geq A[s]}$$

Obviously, after a partition is achieved, $A[s]$ will be in its final position in the sorted array, and we can continue sorting the two subarrays to the left and the right of $A[s]$ independently (e.g., by the same method).

Quick Sort

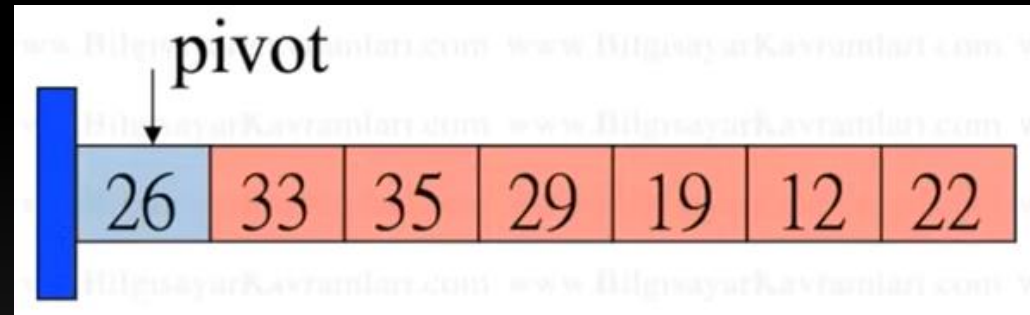


How does it process ?

- We select any element in array as pivot number.
- Generally, it selected first element in original of algorithm. (C.A.R. Hoare, 1962)
- We split up two section left and right side.

Quick Sort

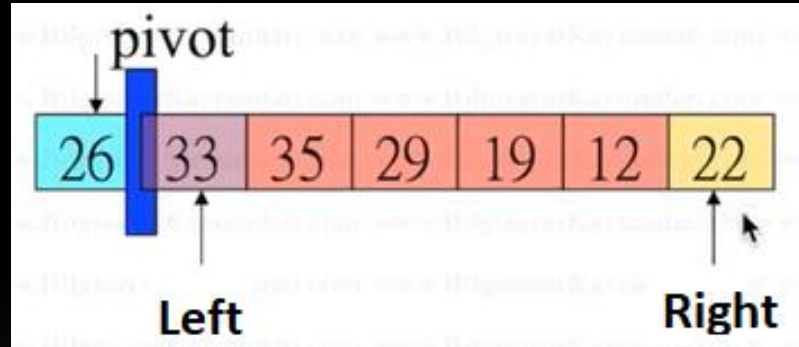
Step I



22, 12, 19, **26**, 33, 35, 29

We move to right side which is smaller than pivot number and move to left side greater than left side.

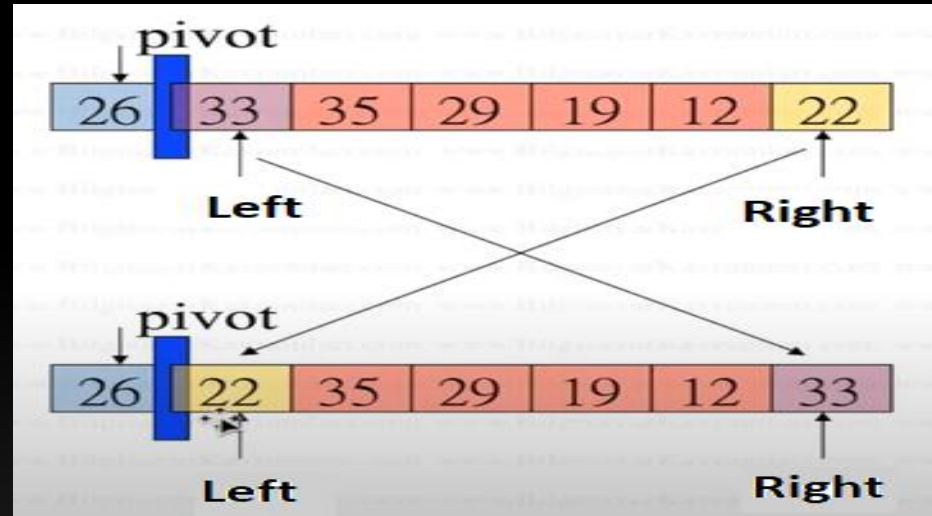
Step 2



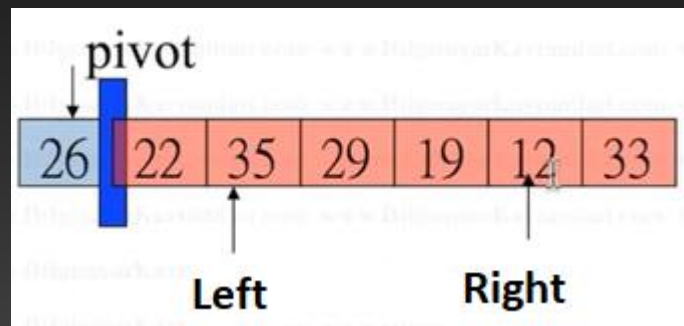
Two pointers are used in order to create left and right groups.

We are checking number at two pivots. Which one is the largest ?

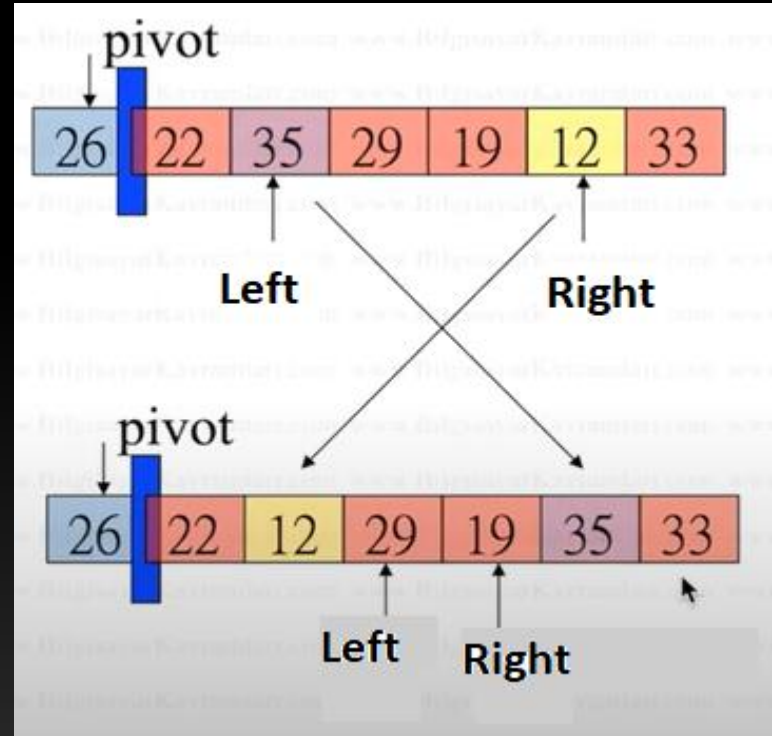
Step 3



We move two pointers after swap processing

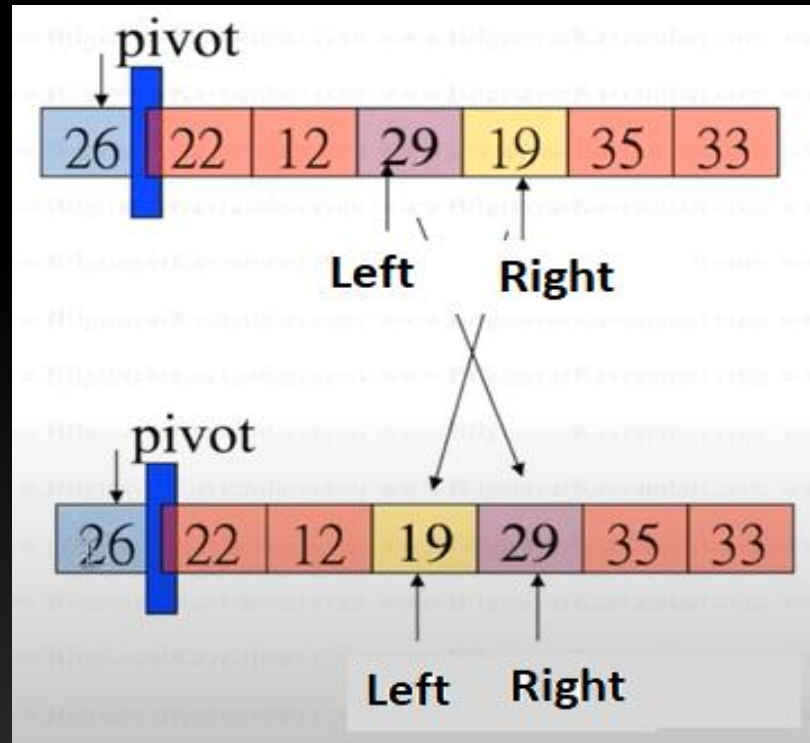


Step 4



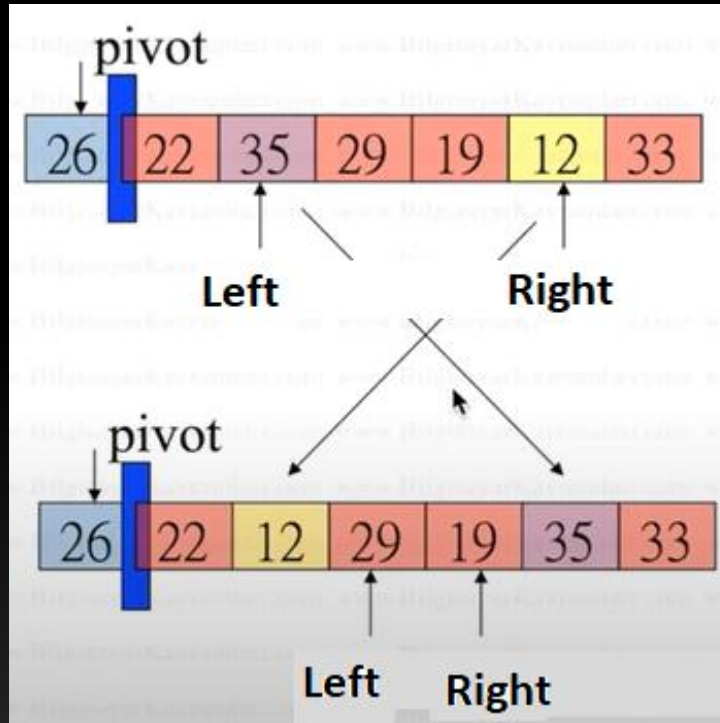
We move two pointers after swap processing

Step 5

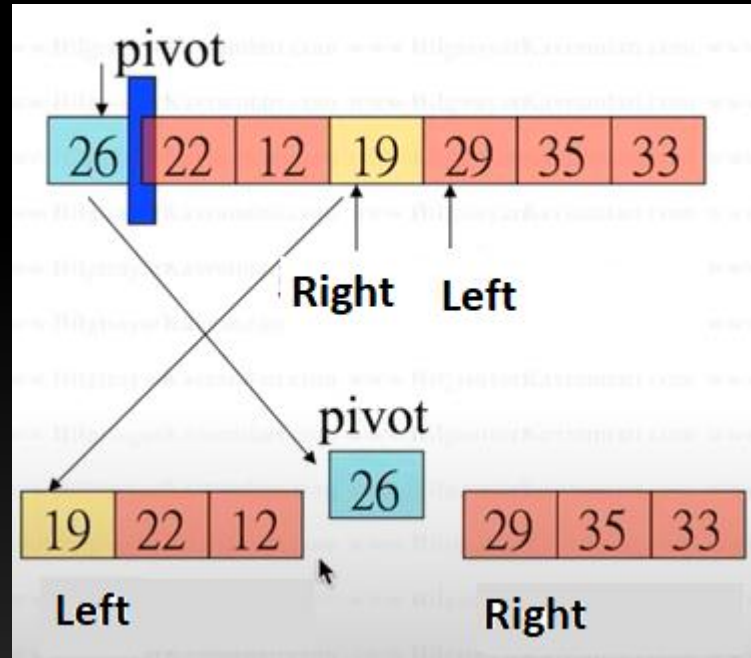


We move two pointers after swap processing

Step 6

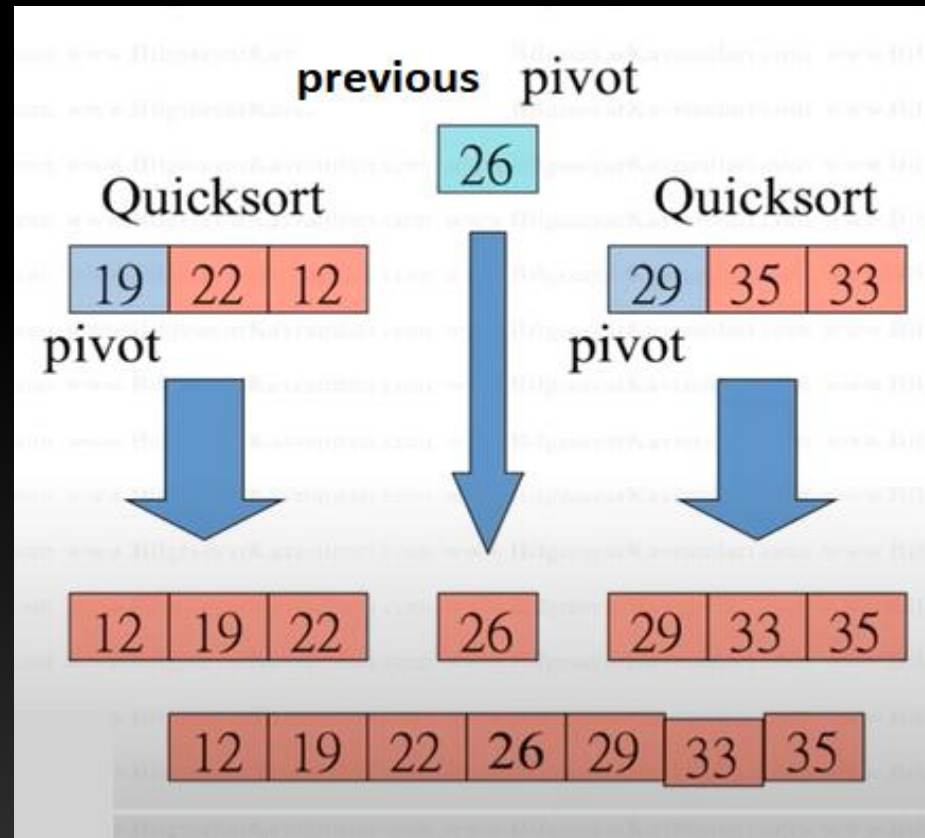


Step 7



It sorted the left and right groups. Finally, it combines the left group, right group and pivot value.

Step 8



The Performance of Quick Sort

The Cost of Break Array: $O(n)$

How many steps does it take to get to a single element? $O(\log_2 n)$

Expected average performance $O(n) = n \cdot \log_2 n$

What is the worst performance ? $O(n) = n^2$

Example: Suppose that we have an array consist of 100 elements.

HOW MANY STEPS DOES IT TAKE TO GET TO A SINGLE ELEMENT ?



The Performance of Quick Sort

Number of checking
elements

N = 100,		100
50+50		100
25+25	25+25	100
12		100
6		100
3		100
1		100

Let's try to create an algorithm for QUICK SORT

```
QuickSort(A, low, high):  
    if low < high:  
        pivot_index = Partition(A, low, high)    // Place pivot in correct  
        QuickSort(A, low, pivot_index - 1)      // Sort left subarray  
        QuickSort(A, pivot_index + 1, high)     // Sort right subarray  
  
Partition(A, low, high):  
    pivot = A[high]    // Select last element as pivot  
    i = low - 1        // i is the index of the last element smaller  
    for j = low to high - 1:  
        if A[j] <= pivot: // If A[j] is smaller or equal to pivot  
            i = i + 1  
            Swap(A[i], A[j]) // Swap A[i] and A[j]  
    Swap(A[i + 1], A[high]) // Place pivot in its correct position  
    return i + 1 // Return the new index of the pivot  
  
Swap(A, i, j):  
    temp = A[i]  
    A[i] = A[j]  
    A[j] = temp
```



Let's try to create a Java code for QUICK SORT

```
public class QuickSortPartition {  
    public static int partition(int A[], int p, int r) {  
        int tmp;  
        int x = A[r]; // Choose the last element as the pivot  
        int i = p - 1; // Index for tracking smaller elements  
  
        for (int j = p; j <= r - 1; j++) {  
            if (A[j] <= x) { // If the current element is smaller than or equal to the pivot  
                i++;  
                // Swap A[i] and A[j]  
                tmp = A[i];  
                A[i] = A[j];  
                A[j] = tmp;  
            }  
        }  
        // Place the pivot in the correct position  
        tmp = A[i + 1];  
        A[i + 1] = A[r];  
        A[r] = tmp;  
  
        return i + 1; // Return the pivot's new position  
    }  
}
```

Let's try to create an algorithm for QUICK SORT

```
}

public static void main(String[] args) {
    int A[] = {10, 80, 30, 90, 40, 50, 70};
    int p = 0;
    int r = A.length - 1;

    int pivotIndex = partition(A, p, r);

    System.out.println("Array after partitioning:");
    for (int num : A) {
        System.out.print(num + " ");
    }
    System.out.println("\nPivot position: " + pivotIndex);
}
}
```

HOMEWORK: Book Sorting – Sorting Books by Height Using Quick Sort

A **library** sorts its books **by height** every year, from the shortest to the tallest. However, this year, new books have mixed with the existing ones.

The librarian needs to **rearrange the books** to restore order.

To do this, the **Quick Sort algorithm** will be used, ensuring the books are sorted by height and placed back on the shelves.



Data Set

Book Title	Height (cm)
Book A	18
Book B	25
Book C	22
Book D	15
Book E	30
Book F	27
Book G	20
Book H	19
Book I	23
Book J	29

If you want to try any sorting method in the app, you can visit this link.



<https://www.hackerearth.com/practice/algorithms/sorting/bubble-sort/practice-problems/>